

ESP32-S3 4848S040 Dual-Core Architecture

LVGL Rendering Engine & Custom Forth VM Integration Strategy

1. Core Allocation & Strategy

On the ESP32-S3, FreeRTOS dedicates Core 0 to Wi-Fi/Bluetooth stacks by default. Isolating the heavy graphical operations of LVGL on Core 1 prevents UI stutter, while running a lower-priority Forth interpreter on Core 0 ensures robust application execution.

Strategy	Core Allocation	Main Benefit	Primary Risk
LVGL Rendering	Core 1 (Dedicated Task)	Smooth UI animations and zero Wi-Fi lag interference.	High PSRAM bandwidth usage if buffers are misconfigured.
Forth VM	Core 0 (Low Priority < 10)	Snappy, real-time command evaluation and control logic.	Stack overflow if shared memory buffers are un-mutexed.

2. Inter-Core Queue Messaging Pattern

Because LVGL is not natively thread-safe, cross-core UI mutations from the Forth VM are routed asynchronously via an internal FreeRTOS queue structure.

Front-End Queue (Core 0 to Core 1): Passes an operation code (Opcode) and packed drawing/widget arguments from Forth. The Core 1 task drains this queue completely before triggering `lv_timer_handler()`.

Back-End Feedback Queue (Core 1 to Core 0): Forwards touch screen events, button clicks, and slider values back to the Forth runtime to match user interaction with script callbacks.

3. Memory Allocation Architecture

- **Internal SRAM:** Reserved for fast execution contexts. Stacks, return stacks, core interpreter loops, and the FreeRTOS messaging queue structures should live entirely inside internal memory to avoid bus contention.
- **External PSRAM:** Utilized for large data allocations. LVGL v8.4 RGB framebuffers (480x480 resolution) and extensive Forth dictionary heaps occupy the slower SPI memory bus.
- **Flash Memory:** Safe for continuous reading of pre-compiled Forth definitions. Direct compilation or data saving to Flash partitions should be heavily guarded to prevent locking up the cache controller shared with the Wi-Fi subsystem.

4. Dynamic Logo-Style Interface Bridge

Integrating a Logo-style canvas configuration gives Forth total programmatic control over vector graphics without binding a heavy C widget infrastructure into the language primitive sets:

1. Create a C lookup registry array mapping unique integer IDs to explicit Forth Execution Tokens (XT addresses).
2. Expose a C-primitive to Forth (e.g., CREATE-BUTTON) that passes coordinates and a target word token to the Core 1 graphics compiler queue.
3. Bind the Forth ID onto the LVGL widget's user data block via `lv_obj_set_user_data()`.
4. Capture the interaction callback inside LVGL, push the ID backward through the feedback queue, and allow the Core 0 engine to issue EXECUTE on the matched XT.